

DevOps — Complete Notes

Table of Contents

1. [Introduction to DevOps](#)
 2. [DevOps Lifecycle \(Stages\)](#)
 3. [DevOps vs Waterfall](#)
 4. [Version Control Systems \(VCS\)](#)
 5. [Git & GitHub Basics](#)
 6. [Continuous Integration / Continuous Delivery / Continuous Deployment \(CI/CD\)](#)
 7. [Docker \(Containerization\)](#)
 8. [Kubernetes \(Orchestration\)](#)
 9. [Infrastructure as Code \(IaC\) — Terraform](#)
 10. [Monitoring — Prometheus & Grafana](#)
 11. [DevSecOps](#)
 12. [Quick-Revision Cheat Sheet](#)
 13. [Common Interview Questions](#)
-

1. Introduction to DevOps

DevOps is a modern software engineering practice that combines **Development (Dev)** and **Operations (Ops)** to build, test, and release software **faster, more frequently, and more reliably**. It emphasizes automation, collaboration, and continuous improvement across the entire software lifecycle.

Why DevOps?

- Encourages close collaboration between development and operations teams (breaks down the traditional "silo" wall).
- Automates build, test, and deployment processes → reduces manual/human errors.
- Enables faster and more frequent software releases.
- Improves system reliability through continuous monitoring and feedback loops.

DevOps vs Traditional Approach

Traditional	DevOps
Dev and Ops work in silos	Dev, Ops, and QA collaborate continuously
Manual deployments	Automated CI/CD pipelines
Infrequent, large releases	Small, frequent releases
Errors found late	Errors caught early (shift-left testing)

2. DevOps Lifecycle (Stages)

The DevOps lifecycle is usually visualized as an **infinite loop** because it never truly "ends" — feedback from monitoring feeds back into planning.

2.1 Plan

Defining project goals, requirements, and breaking down tasks/user stories so Dev and Ops align from day one.

- **Tools:** Jira, Confluence, Azure Boards, Trello

2.2 Code

Writing, reviewing, and managing source code and configuration using version control.

- Code reviews and branching strategies maintain code quality and stability.
- **Tools:** Git, GitHub, GitLab, Bitbucket

2.3 Build

Automatically compiling and packaging code into deployable artifacts while resolving dependencies.

- Build automation gives faster feedback and reduces manual errors.
- **Tools:** Jenkins, GitLab CI/CD, Maven, Gradle, Docker

2.4 Test

Running automated quality, security, and performance checks to catch bugs **before release**.

- Types: unit, integration, performance, and security testing.
- Catching issues early reduces the cost/impact of failures ("shift-left").
- **Tools:** Selenium, JUnit, TestNG, SonarQube, JMeter

2.5 Release

Finalizing and documenting approved builds through version tagging and release-strategy planning.

- Deployment strategies are planned in advance to minimize production risk.
- **Tools:** Git tags, Jenkins, GitLab CI/CD, ArgoCD

2.6 Deploy

Pushing the application into production using automated infrastructure and rollout strategies.

- Strategies: **blue-green**, **canary**, or **rolling updates** — all designed to minimize downtime.
- **Tools:** Kubernetes, Helm, Ansible, Terraform

2.7 Operate & Monitor

Maintaining system health and gathering real-world performance data to drive continuous improvement.

- Continuous monitoring ensures availability and performance.
- Logs, metrics, and alerts help detect and resolve issues quickly.
- Feedback loops back into the **Plan** stage — completing the cycle.
- **Tools:** Prometheus, Grafana, ELK Stack, Datadog, New Relic

How to Adopt DevOps Strategically

1. **Foster a unified mindset** — cross-functional collaboration and shared accountability (Dev + Sec + Ops).
2. **Assess infrastructure needs** — find bottlenecks, define scalability/security requirements.
3. **Define measurable goals** — deployment speed, reliability, team alignment.
4. **Select integrated toolsets** — CI/CD, version control, monitoring tools that fit your stack.
5. **Prioritize automated testing** — shift QA left into the dev cycle.
6. **Standardize with containers** — ensures environment consistency.
7. **Iterate and optimize** — use monitoring/feedback loops to refine continuously.

Role of AI/ML in DevOps

- **Smart data analysis** — scans huge volumes of code/test data to highlight what matters.

- **Automatic shortcuts** — ML learns team patterns and suggests faster server setup / repetitive-task automation.
 - **Predictive problem detection** — spots unusual code patterns to flag bugs *before* they cause a crash.
 - **Instant security guard** — 24/7 anomaly detection that can auto-block suspicious activity.
-

3. DevOps vs Waterfall

DevOps	Waterfall
Continuous development & deployment	Step-by-step, rigid process
Dev, Ops, and QA work together	Teams work separately
Rapid, frequent releases	Slow, long release cycles
High automation (CI/CD, testing, monitoring)	Mostly manual processes
Easily adapts to changes	Hard to modify once planned
Continuous monitoring, early issue detection	Errors found late in the cycle

4. Version Control Systems (VCS)

A **Version Control System** tracks and manages changes to source code so teams can collaborate without overwriting each other's work.

Core Concepts

Term	Meaning
Repository	Central location storing all project files + full change history/metadata
Revision	A specific saved version of a file/project, identified by a hash or number
Branch	A separate copy of the codebase to build features/fix bugs independently
Merging	Combining changes from one branch into another (may need conflict resolution)
Commit	A snapshot of changes at a specific point in time

4.1 Types of VCS

a) Local Version Control Systems

- Stores all versions on a single machine; no server/internet dependency.
- Good for solo projects; not usable for team collaboration.

b) Centralized Version Control Systems (CVCS)

- All files + history live on **one central server**.
- Workflow: **Update/Checkout** → **Make Changes** → **Commit** (directly to server).
- **Pros:** Central visibility, fine-grained access control.
- **Cons:** Single point of failure — if the server goes down, nobody can commit.
- Example: **SVN (Subversion)**, CVS

c) Distributed Version Control Systems (DVCS)

- Every developer has a **full local repository** + working copy.
- Workflow: **Commit** (local only) → **Push** (upload to shared repo) → **Pull** (download others' changes).
- Two-step process (commit → push) is the key differentiator from CVCS.

- Example: **Git**, Mercurial, Bazaar

4.2 Popular VCS Tools

Tool	Type	Notes
Git	Distributed	Created by Linus Torvalds (2005) for Linux kernel dev. Lightweight, fast, powerful branching/merging. Backbone of GitHub/GitLab/Bitbucket.
Subversion (SVN)	Centralized	Still used in enterprises for its simplicity
Mercurial	Distributed	Similar to Git, simpler interface
CVS	Centralized	Legacy tool; foundation for SVN
Bazaar	Both	Made by Canonical (Ubuntu); beginner-friendly

5. Git & GitHub Basics

Git is the industry-standard distributed VCS. Key commands every DevOps engineer should know:

Command	Purpose
<code>git init</code>	Initialize a new local repository
<code>git clone <url></code>	Copy a remote repository locally
<code>git add <file></code>	Stage changes for commit
<code>git commit -m "msg"</code>	Save a snapshot of staged changes
<code>git push</code>	Upload local commits to remote repo
<code>git pull</code>	Fetch + merge remote changes into local branch

<code>git branch <name></code>	Create a new branch
<code>git checkout <branch></code> <code>/git switch</code>	Switch branches
<code>git merge <branch></code>	Merge another branch into current one
<code>git rebase</code>	Reapply commits on top of another base branch (linear history)
<code>git status</code>	Show current state of working directory
<code>git log</code>	View commit history

Branching Strategies: Git Flow, GitHub Flow, trunk-based development — used to organize how features/releases/hotfixes move through branches.

Merge vs Rebase: Merge preserves full history with a merge commit; rebase rewrites commit history to appear linear (cleaner logs, but rewrites shared history — use cautiously on shared branches).

6. Continuous Integration / Continuous Delivery / Continuous Deployment (CI/CD)

CI/CD automates the process of building, testing, and releasing applications — a central pillar of DevOps.

Life *Before* CI/CD

- All branches merged at the very end → big conflicts, broken builds.
- Testing/building happened manually at the final stage → bugs found late & costly to fix.
- Deployment took days/weeks since everything shipped in one giant release.

- Dev, Test, and Ops teams worked in isolated silos.

Life *After* CI/CD

- Developers commit code frequently to a shared repo.
- CI automatically builds and tests every commit.
- Bugs are caught and fixed early.
- Continuous Delivery keeps code always release-ready.
- Continuous Deployment automatically ships to production.
- Smaller, frequent updates replace large risky releases.

6.1 The Three Pillars

1. Continuous Integration (CI)

- **Goal:** Prevent "integration hell" from late code merges.
- Developers merge into the main branch frequently (often daily).
- Every commit triggers an automated build + unit tests.
- If tests fail → build rejected, developer notified immediately.

2. Continuous Delivery (CD)

- **Goal:** Keep the codebase always in a release-ready state.
- After CI passes, code auto-deploys to a **staging/test** environment.
- Integration, system, and performance tests run automatically.
- **Production release is manual** — a human clicks "deploy."

3. Continuous Deployment (CD)

- **Goal:** Fully automated, hands-off production releases.
- After all tests pass, code is **automatically** deployed to production — no human step.
- Requires very mature, comprehensive automated test coverage.

△ **Key interview distinction:** Continuous **Delivery** = manual approval before prod. Continuous **Deployment** = fully automatic, no approval gate.

6.2 CI/CD Pipeline — Component Breakdown

Stage	What Happens
1. Commit Change	Developer pushes code to Git; changes are tracked/versioned
2. Build Trigger	VCS detects the new commit and auto-starts the pipeline
3. Build	Code compiled & packaged into a deployable artifact; dependencies resolved (Maven/Gradle/Docker)
4. Build Outcome Notification	Team notified pass/fail
5. Run Execution (Test)	Unit, integration, end-to-end tests run automatically
6. Test Outcome Notification	Team notified of test results for quick debugging
7. Deliver to Staging	Deployed to a production-like environment for final validation
8. Deploy to Production	Application released to end users

6.3 Common CI/CD Tools

- **Jenkins** — open-source automation server (most widely used, plugin-based)
- **GitHub Actions** — native CI/CD built into GitHub
- **GitLab CI/CD** — built-in pipeline solution within GitLab
- **Spinnaker** — multi-cloud continuous delivery platform
- **GoCD, Concourse, Screwdriver** — other pipeline automation tools

6.4 Best Practices for a Healthy Pipeline

- **Fast feedback** — detect failures quickly.
 - **Commit frequently** — avoid large, conflict-prone merges.
 - **Fix broken builds immediately** — keep `main` always stable/deployable.
 - **Environment parity** — staging should mirror production closely.
 - **Use IaC** — Terraform/CloudFormation to keep infra consistent across environments.
-

7. Docker (Containerization)

Docker is an **OS-level virtualization** platform. Containers share the host OS kernel (unlike VMs, which each need a full guest OS), making them lightweight, fast to start, and portable.

The Problem Docker Solves

Before Docker: "works on my machine" — differences in dependencies, library versions, and OS configs broke deployments across dev/test/prod.

Docker's fix: bundle the app code + its exact dependencies into one standardized unit that runs identically everywhere.

Key Benefits

- **Portability** — runs anywhere: laptop, on-prem, cloud.
- **Consistency** — identical behavior across dev/test/prod.
- **Lightweight** — no full OS per app; containers share the host kernel.
- **Scalability** — ideal for microservices + orchestrators (Kubernetes, Swarm).
- **Efficiency** — starts in seconds, uses far fewer resources than a VM.

7.1 Docker Architecture (Client–Server Model)

- 1. **Docker Client (CLI)** — how users interact with Docker (`docker run`, etc.). Sends commands to the daemon via REST API.
- 2. **Docker Daemon (`dockerd`)** — background service that manages images, containers, networks, and volumes.
- 3. **Docker Registry (Docker Hub)** — storage for images; the largest public registry for pulling pre-built images (Ubuntu, MySQL, Nginx, etc.).

Communication happens over a **REST API** (UNIX socket or network).

7.2 Key Components

Component	Description
Docker Engine	Core runtime; the daemon that creates/manages containers
Dockerfile	Text file with step-by-step instructions (DSL) to build an image
Docker Image	Read-only template/blueprint containing app code + dependencies
Docker Container	A running instance of an image (dynamic, executable)
Docker Hub	Cloud-based registry to find/share container images
Docker Registry	Storage/distribution system for images (public or private)

Image vs Container (critical distinction):

Image = Blueprint (static, read-only) → **Container** = Live running instance of that blueprint (dynamic, executable).

7.3 Dockerfile

A Dockerfile uses a **Domain Specific Language (DSL)**. The Docker daemon executes instructions **top to bottom** to build the image.

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

7.4 Essential Docker Commands

Command	Purpose
<code>docker run</code>	Launch a container from an image
<code>docker pull</code>	Fetch an image from a registry (e.g., Docker Hub) to local machine
<code>docker ps</code>	List running containers (ID, image, status)
<code>docker stop</code>	Gracefully halt a running container
<code>docker start</code>	Restart a stopped container
<code>docker login</code>	Authenticate to access private registries
<code>docker build -t <name> .</code>	Build an image from a Dockerfile
<code>docker images</code>	List local images
<code>docker exec -it <container> bash</code>	Open a shell inside a running container

7.5 Docker Editions

1. **Community Edition (CE)** — free, open-source; for individuals &

dev teams.

2. **Enterprise Edition (EE)** — paid; adds security features, certified plugins/images, enterprise support.
-

8. Kubernetes (Orchestration)

The Problem Kubernetes Solves

Docker/Docker Swarm handle **packaging** an app into containers. But once you have hundreds/thousands of containers, you hit new problems:

- Scalability issues
- Multi-cloud deployment complexity
- Security & resource management at scale
- Achieving rolling updates with **zero downtime**

Kubernetes (K8s) is the "orchestrator" — the brain that automates deployment, scaling, and management of containerized apps at scale.

Origin & Facts

- Developed by **Google**, inspired by internal systems **Borg** and **Omega**.
- Released in **2014**; donated to **CNCF** (Cloud Native Computing Foundation) in **2015**.
- Name is Greek for "**helmsman**" / "**pilot**" — reflecting its role steering applications.
- "K8s" = K + 8 letters + s.

Analogy: Kubernetes is like an **orchestra conductor**. Each container is a musician — you give the conductor the sheet music (your desired config), and it ensures every musician plays correctly, swapping out anyone who "falls ill" (fails) automatically.

Key Features

- **Automated Scheduling** — places containers on nodes for optimal resource use.
- **Self-Healing** — auto-restarts/replaces/reschedules failed containers.
- **Rollouts & Rollbacks** — manages updates and reverts if something breaks.
- **Scaling & Load Balancing** — horizontal scaling + traffic distribution.
- **Resource Optimization** — continuously monitors resource utilization.

8.1 Monolithic vs Microservices

- **Monolithic:** everything bundled into one big codebase. Changing one module (e.g., payments) requires redeploying the *entire* app — risky, and a small bug can crash the whole system.
- **Microservices:** each feature (payments, search, notifications) is built and deployed **independently** — more flexible and scalable.
- **New challenge:** now you're managing hundreds/thousands of small containerized services — this is exactly what Kubernetes automates and coordinates.

8.2 Core Kubernetes Terminology

Term	Description
Pod	Smallest deployable unit in K8s. Wraps one or more containers that share network + storage and run together.
Node	A physical/virtual machine in the cluster that runs Pods. Contains container runtime, Kubelet, and Kube-proxy.

Cluster	A group of Nodes working together. Has a Master Node (Control Plane) — the "brain" that schedules and tracks everything — and Worker Nodes that actually run the apps.
Deployment	A K8s object managing a set of Pods; provides <i>declarative</i> updates (you say what you want, K8s figures out how).
ReplicaSet	Ensures the correct number of identical Pods are always running.
Service	Gives Pods a stable way to communicate even as individual Pods come and go.
Ingress	Manages external HTTP/HTTPS access to services — acts as a reverse proxy.
ConfigMap	Stores configuration settings (e.g., DB connection strings) separately from app code, so config can change without redeploying code.
Secret	Securely stores sensitive data (passwords, API keys, tokens).
Persistent Volume (PV)	Storage that survives Pod deletion/restart.
Kubelet	Agent running on each Worker Node ensuring Pods run as expected.
Kube-proxy	Manages cluster networking so Pods can communicate.

8.3 Cluster Structure

Kubernetes Cluster

- ├─ Master Node (Control Plane) — scheduling & decision-making
- └─ Worker Nodes — each runs:
 - ├─ Kubelet (agent)
 - ├─ Container Runtime (Docker/containerd)
 - └─ Kube-proxy (networking)

8.4 Kubernetes vs Docker

Docker packages and runs individual containers; Kubernetes **orchestrates many containers across many machines** — scheduling, scaling, self-healing, and networking them as a system. They're complementary, not competitors: Kubernetes typically runs Docker (or another runtime) *underneath* it.

9. Infrastructure as Code (IaC) — Terraform

Infrastructure as Code (IaC) manages IT infrastructure using **configuration files** instead of manual, click-through console setup.

- **Declarative:** you state *what* you want ("5 servers"), and the tool figures out *how*.
- **Version-controlled:** infra changes tracked in Git just like application code.

Terraform (by HashiCorp) is the industry-standard IaC tool for provisioning infrastructure safely and repeatably.

Key Features

1. **Cloud Agnostic** — works with AWS, GCP, Azure, Kubernetes, Alibaba, etc. (unlike CloudFormation, which is AWS-only).
2. **Immutable Infrastructure** — typically *replaces* servers rather than mutating them → avoids "configuration drift."
3. **State Management** — tracks real-world resources in a **state file** (the "source of truth").
4. **Modular** — reusable **Modules** package common infra patterns (e.g., a standard "web server" module).

9.1 Architecture

Component	Role
Core (Engine)	Reads config files, compares to current state, calculates required changes
Providers	Plugins that translate Terraform code into API calls for a specific platform (AWS, Azure, Kubernetes providers, etc.)
State File (terraform.tfstate)	Maps your code to real-world resources; the "brain" of Terraform. In teams, stored remotely (e.g., S3) so everyone shares the same source of truth.

9.2 Core CLI Commands

Command	Purpose
terraform init	Initializes working directory; downloads required providers
terraform validate	Checks configuration syntax validity
terraform plan	Shows a preview of changes — does NOT apply them
terraform apply	Executes the plan — creates/modifies real infra
terraform destroy	Deletes all resources tracked in the state file
terraform import	Brings an existing resource under Terraform management
terraform console	Interactive console for evaluating expressions
terraform refresh	Syncs state file with actual real-world infra

9.3 HCL (HashiCorp Configuration Language) Example

```
# 1. Define the Provider
provider "aws" {
  region = "us-east-1"
}

# 2. Define a Resource
resource "aws_instance" "my_web_server" {
  ami           = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "DevOps-Server"
  }
}
```

- `resource` → keyword to define infrastructure.
- `aws_instance` → resource **type** (from AWS provider plugin).
- `my_web_server` → internal name Terraform uses to track this resource.
- Block contents { } → resource properties/arguments.

9.4 State: Local vs Remote

	Local State	Remote State
Setup	Simple, no backend config	Needs a backend (S3, Terraform Cloud, Azure Blob, GCS)
Best for	Learning, solo projects	Team/production environments
Risk	No locking → conflict-prone; risk of accidental deletion	Supports state locking , prevents simultaneous edits

Collaboration	Hard to share safely	Centralized, secure, shareable
----------------------	----------------------	--------------------------------

9.5 Terraform vs Other IaC Tools

Terraform vs AWS CloudFormation

Feature	Terraform	CloudFormation
Scope	Multi-cloud	AWS only
Language	HCL (clean, simple)	JSON/YAML (verbose)
State	Managed by user	Managed automatically by AWS

Terraform vs Ansible

Feature	Terraform	Ansible
Primary Use	Provisioning infrastructure	Configuring systems / deploying apps
Language	HCL	YAML
Execution	Uses state + plans	Executes tasks immediately, no state tracking
Cloud Support	Excellent multi-cloud	Good, but more system-level focused

10. Monitoring — Prometheus & Grafana

10.1 Prometheus

Prometheus is an open-source monitoring tool that captures and stores **time-series data** (metrics + labels + timestamps).

Why Prometheus?

- Real-time visibility into system performance.
- Early failure detection.
- Improved operational reliability.
- Strong fit for cloud-native / microservices architectures.

Core Components

Component	Role
Prometheus Server	Central component; scrapes and stores metric data in a local TSDB
Targets	Endpoints/services being monitored (discovered dynamically or statically configured)
Exporters	Expose metrics in Prometheus format (e.g., Node Exporter for hardware/OS metrics, MySQL/ Apache exporters)
PromQL	Prometheus Query Language — for retrieving/ analyzing time-series data
Alertmanager	Handles alerts: deduplication, grouping, routing to Slack/email/PagerDuty
TSDB	Time-Series Database — where all metrics are stored with timestamps + key-value labels

How it Works (Pull Model)

- Prometheus **pulls/scrapes** metrics from targets over HTTP on a schedule (rather than targets pushing data to it).
- Supports **service discovery** (e.g., auto-discovering Kubernetes pods) so new targets are monitored without manual config.
- Metrics are stored as **(name + labels + timestamp + value)** tuples.
- Alert rules are evaluated continuously; matches trigger Alertmanager notifications.

Metric Types

Type	Description
Counter	Value that only goes up (e.g., total requests served); can reset to 0
Gauge	Value that can go up or down (e.g., current memory usage, active connections)
Summary	Tracks total count + sum of observed values; computes quantiles client-side
Histogram	Buckets observations (e.g., response times); quantiles computed server-side

Advantages

- Open-source, vendor-neutral, cloud-native by design.
- Powerful querying (PromQL).
- Native Kubernetes integration.
- Reliable alerting.

Limitations

- Not ideal for **long-term** storage.
- Primarily a **single-node** system.
- Needs external tools (like Grafana) for rich visualization.

10.2 Grafana

Grafana is a visualization tool commonly paired with Prometheus to build dashboards from metrics data (graphs, panels, alert views). While Prometheus collects and stores metrics, Grafana turns them into readable, shareable dashboards.

11. DevSecOps

DevSecOps extends DevOps by embedding **security** into every stage of the pipeline — rather than treating it as a final gate before release

("shift-left security").

- Security checks (SAST, DAST, dependency scanning) are automated **within** the CI/CD pipeline.
 - Encourages shared responsibility for security across Dev, Sec, and Ops teams.
 - Common tools: SonarQube (code quality/security scanning), Snyk, Aqua Security, container image scanners.
-

12. Quick-Revision Cheat Sheet

Concept	One-Line Definition
DevOps	Culture + practices unifying Dev and Ops for faster, reliable software delivery
CI	Frequently merging + auto-testing code to catch issues early
CD (Delivery)	Auto-deploy to staging; manual approval to go to production
CD (Deployment)	Fully automatic deployment to production, no human gate
Docker Image	Static, read-only blueprint
Docker Container	Running, live instance of an image
Kubernetes Pod	Smallest deployable unit; one or more containers sharing network/storage
Kubernetes Service	Stable network endpoint for a set of changing Pods
Terraform Plan	Preview of changes (no execution)
Terraform Apply	Actually executes changes on real infra
IaC	Managing infra via version-controlled code, not manual console clicks

Prometheus	Pull-based metrics collection + time-series storage
Grafana	Visualization layer on top of metrics (often paired with Prometheus)
Blue-Green Deployment	Two identical environments; switch traffic instantly to the new one
Canary Deployment	Roll out to a small subset of users first, then expand gradually
Rolling Update	Gradually replace old instances with new ones, one/few at a time

--